

FECAP

Fundação Escola de Comércio Álvares Penteado

Curso de Ciência da Computação
Disciplina: Inteligência Artificial e Aprendizado de Máquina

ENTREGA 1

Lideranças Empáticas

Contagem Automática de Alimentos Doados

Integrantes:

Felipe Vallim Soares — RA: 24026060
Guilhermy Mariano Lisboa Garcia — RA: 23025371
Gustavo Oliveira Demetrio — RA: 24026213
Saulo Pereira de Jesus — RA: 24026095

1. INTRODUÇÃO

Requisitos da Entrega

- Identificação visual de 3 produtos diferentes
- Formato de Entrega: Jupyter Notebook / Código Python
- Objetivo: Implementar e treinar um modelo de IA para uma finalidade

O presente projeto tem como objetivo o desenvolvimento de um sistema de visão computacional capaz de detectar, classificar e contar automaticamente embalagens de alimentos doados em um ambiente físico controlado. O cenário de captura consiste em uma rampa com fundo escuro, iluminação fixa e câmera posicionada sobre a rampa, garantindo condições padronizadas para a aquisição de imagens.

O sistema identifica três categorias de alimentos: arroz (classe 0), feijão (classe 1) e outros (classe 2, que engloba açúcar, café e demais itens). Essa classificação atende ao requisito da disciplina de identificação visual de 3 produtos diferentes.

O modelo escolhido foi o YOLOv8n (nano) da Ultralytics, que realiza detecção e classificação em um único passo. Diferentemente de classificadores tradicionais como o MobileNetV2, que apenas classificam imagens sem fornecer bounding boxes, o YOLO é capaz de localizar e identificar objetos simultaneamente, sendo mais adequado para aplicações de contagem em tempo real.

Esta entrega inclui: Jupyter Notebook documentado com o pipeline completo, scripts Python modulares, modelo treinado (.pt), métricas de avaliação detalhadas e demonstração em tempo real via webcam.

2. METODOLOGIA

2.1 Dataset

A base de dados foi construída com 88 imagens capturadas em ambiente controlado, distribuídas em três categorias: arroz (17 imagens), feijão (28 imagens) e outros (43 imagens).

Para expandir o dataset, foi implementado um pipeline de Data Augmentation utilizando a biblioteca OpenCV. Cada imagem original gera 1 cópia redimensionada (640×640) e 10 variantes aumentadas, resultando em um total de 968 imagens. As técnicas aplicadas estão detalhadas na tabela a seguir:

Técnica	Parâmetros	Probabilidade	Objetivo
Rotação	±20 graus	70%	Variação de ângulo
Flip	Horizontal, vertical, ambos	50%	Espelhamento
Brilho/Contraste	alpha [0.8, 1.2], beta [-30, 30]	50%	Variação de iluminação
Blur Gaussiano	kernel 3x3 ou 5x5	30%	Suavização
Ruído Gaussiano	sigma = 10	30%	Robustez a ruído
Zoom	escala [0.85, 1.15]	50%	Variação de escala

A divisão entre conjuntos de treino e validação foi realizada com proporção 80/20 e seed 42, resultando na seguinte distribuição:

Conjunto	Arroz	Feijão	Outros
Treino (80%)	150	247	379
Validação (20%)	55	99	159
Total	205	346	538

Os labels foram gerados no formato YOLO (<classe> <x_centro> <y_centro> <largura> <altura>) com bounding box fixa em 0.5 0.5 0.8 0.8, considerando que os itens são centralizados na rampa de captura.

2.2 Arquitetura do Modelo

O modelo utilizado é o YOLOv8n (nano) da Ultralytics, pré-treinado no dataset COCO e retreinado com transfer learning para as três classes do projeto. Suas características técnicas incluem:

- 73 camadas com 3.006.233 parâmetros treináveis
- 8.1 GFLOPs de complexidade computacional
- Tamanho do modelo: aproximadamente 6.3 MB
- Velocidade de inferência: 42–45 ms por frame em CPU (13th Gen Intel Core i5-13450HX)

2.3 Hiperparâmetros de Treinamento

O treinamento foi configurado com os seguintes hiperparâmetros:

Parâmetro	Valor
Épocas	30
Batch size	8
Tamanho de imagem	640 × 640 pixels
Modelo base	yolov8n.pt (pré-treinado COCO)
Framework	Ultralytics 8.4.24 + PyTorch 2.10.0
Hardware	CPU — 13th Gen Intel Core i5-13450HX
Tempo de treinamento	~57 minutos (0.941 horas)

2.4 Lógica de Contagem (Inferência)

A contagem de alimentos durante a inferência em tempo real é realizada por meio de uma lógica de estabilidade. Para que um item seja contabilizado, ele precisa ser detectado por 10 frames consecutivos com confiança mínima de 0.75. Após a contagem, um período de cooldown de 3 segundos é aplicado para evitar duplicatas.

Caso nenhum objeto seja detectado em um frame, o contador de estabilidade é resetado. O threshold de 0.75 foi escolhido para eliminar falsos positivos no fundo sem comprometer as detecções reais, enquanto os 10 frames estáveis previnem contagens espúrias.

3. RESULTADOS E MÉTRICAS

As métricas a seguir foram obtidas na avaliação do modelo treinado sobre o conjunto de validação (313 imagens):

Métricas Gerais

Métrica	Valor
mAP50	0.6337
mAP50-95	0.6337
Precision geral	0.543
Recall geral	0.985

Métricas por Classe

Classe	Precision	Recall	AP50
Arroz (classe 0)	0.5217	0.9545	0.6878
Feijão (classe 1)	0.5167	1.0000	0.5776
Outros (classe 2)	0.5901	1.0000	0.6358

Evolução do Treinamento

Loss	Época 1	Época 30	Redução
box_loss	1.199	0.316	73.6%
cls_loss	2.408	0.240	90.0%
dfl_loss	1.741	1.143	34.4%

Análise dos Resultados

O Recall obtido foi excelente, variando de 0.9545 a 1.0000 em todas as classes, o que indica que o modelo detecta praticamente todos os itens presentes nas imagens. A Precision moderada (em torno de 0.52 a 0.59) aponta a presença de falsos positivos, porém esse fator é mitigado na aplicação prática pela lógica de estabilidade implementada (10 frames consecutivos com threshold de 0.75).

A evolução das funções de perda demonstra convergência efetiva: a cls_loss (classificação) apresentou redução de 90%, a box_loss (localização) reduziu 73.6%, e a dfl_loss (distribuição focal) reduziu 34.4%. Os gráficos de curvas (Precision, Recall, F1 e PR) e matrizes de confusão foram gerados automaticamente e estão disponíveis na pasta runs/detect/.

4. TECNOLOGIAS UTILIZADAS

O desenvolvimento do projeto utilizou o seguinte conjunto de tecnologias:

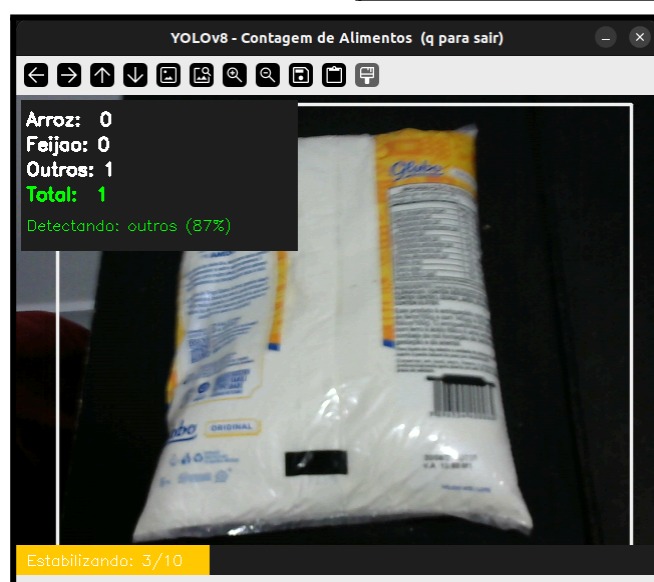
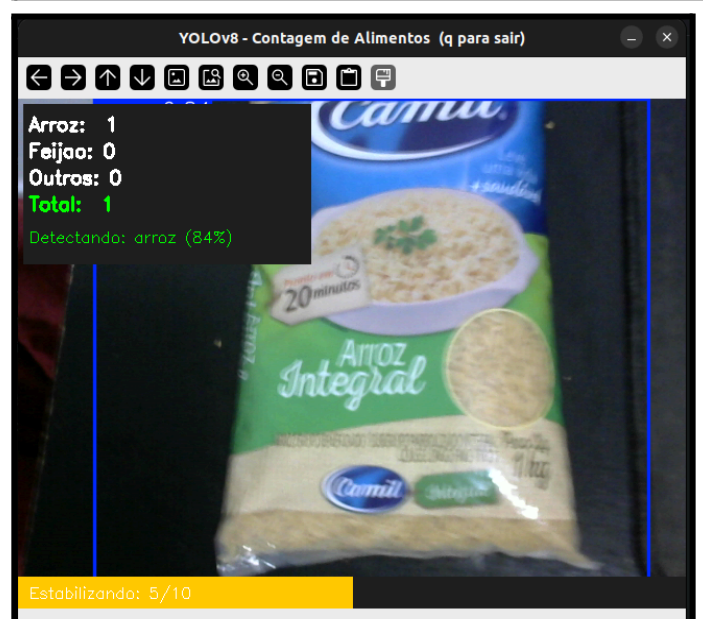
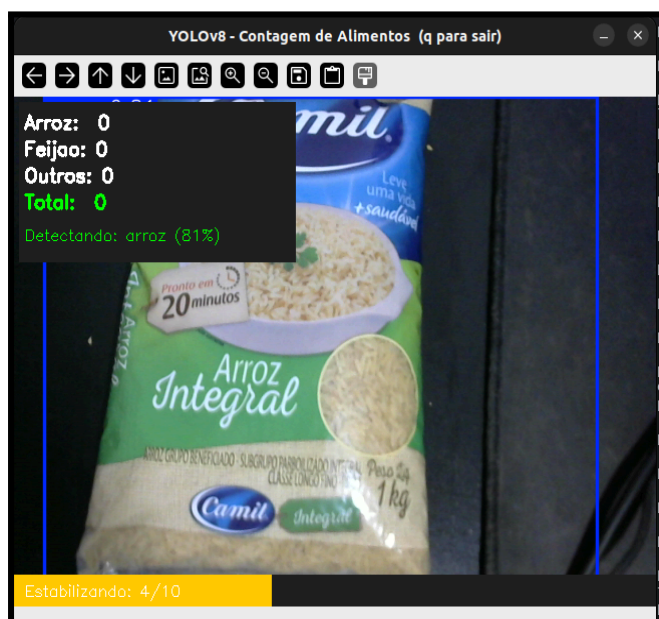
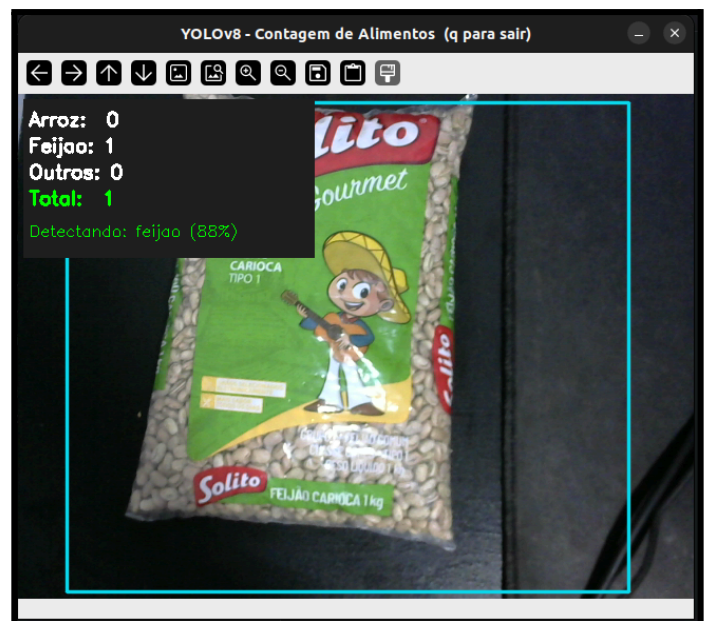
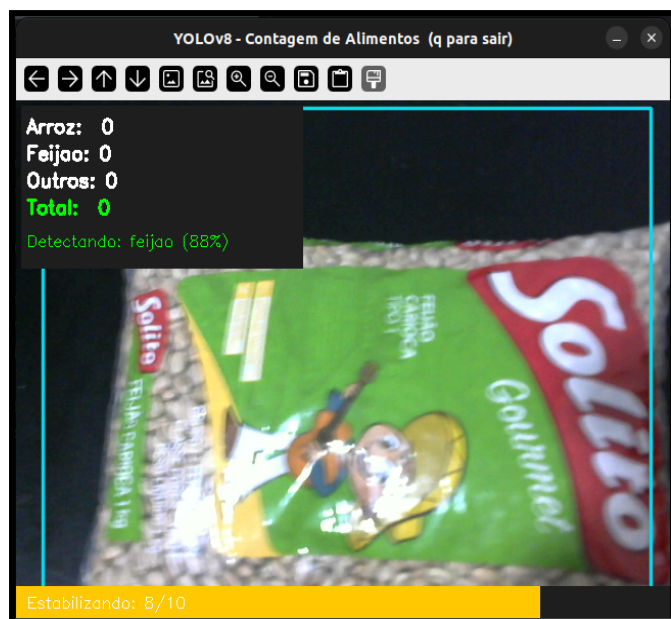
Tecnologia	Finalidade
Python 3.10	Linguagem principal do projeto
YOLOv8 (Ultralytics 8.4.24)	Deteção de objetos em tempo real
PyTorch 2.10.0	Framework de deep learning
OpenCV	Processamento de imagem e data augmentation
NumPy	Manipulação de arrays e operações numéricas
Jupyter Notebook	Documentação interativa do pipeline
FastAPI	API REST para integração (backend)
React + TypeScript	Frontend (integração futura)

5. ESTRUTURA DE ARQUIVOS DA ENTREGA

A tabela a seguir descreve os arquivos e diretórios que compõem a entrega:

Arquivo	Descrição
food_classifier.ipynb	Notebook principal com pipeline completo
config.py	Hiperparâmetros e configurações centralizadas
generate_dataset.py	Script de data augmentation com OpenCV
split_dataset.py	Divisão train/val 80/20 com seed 42
generate_labels.py	Geração de labels no formato YOLO
train_yolo.py	Script de treinamento do YOLOv8n
evaluate.py	Avaliação de métricas do modelo treinado
webcam_demo.py	Demo de inferência em tempo real via webcam
data.yaml	Configuração do dataset para YOLO
yolov8n.pt	Modelo base pré-treinado (COCO)
models/best.pt	Modelo treinado final (6.3 MB)
dataset_base/	88 imagens originais organizadas por categoria
dataset/	Dataset aumentado com split train/val
runs/detect/	Resultados de treinamento, métricas e gráficos

6. EVIDÊNCIAS DE EXECUÇÃO — WEBCAM EM TEMPO REAL



7. CONCLUSÃO

O objetivo desta primeira entrega foi plenamente atingido: o modelo foi treinado com sucesso para identificar visualmente três categorias de alimentos doados (arroz, feijão e outros), atendendo ao requisito de identificação visual de 3 produtos diferentes.

O pipeline completo foi implementado de forma modular, abrangendo todas as etapas do processo: coleta e organização das imagens, data augmentation para expansão do dataset, treinamento do modelo com transfer learning, avaliação quantitativa por métricas padronizadas e inferência em tempo real via webcam.

O Recall elevado (0.95 a 1.00 em todas as classes) garante que o sistema detecta praticamente todos os itens presentes, enquanto a lógica de estabilidade (10 frames consecutivos com threshold de 0.75) compensa a Precision moderada, assegurando confiabilidade na contagem durante a aplicação prática.

Próximos Passos

- Expandir o dataset com mais imagens e novas categorias de alimentos
- Integração completa com frontend e backend FastAPI
- Armazenamento de evidências e dados de contagem em nuvem
- Otimização do modelo para melhorar a Precision sem comprometer o Recall